MODULE 3

MACHINE LEARNING PRACTICALS AND MODEL IMPLEMENTATION

Loan Default Prediction System
A Comparative Study of Classification Algorithms

Submitted by: Advik Singh

Registration Number: RA2411056030023

Branch / Section: DS-A

Course: Machine Learning / Data Science and Machine Learning Training

Abstract

Banks approve loans every day without knowing for certain who will repay and who will default. This project builds a predictive system that estimates the risk of loan default from three customer attributes — employment status, bank balance, and annual salary — using a banking dataset of 10,000 customers. Nine classification algorithms (Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, K-Nearest Neighbours, Gradient Boosting, AdaBoost, XGBoost, and CatBoost) are trained with appropriate handling of class imbalance, and compared using accuracy, precision, recall, F1-score, and ROC-AUC. Hyperparameter tuning via GridSearchCV is demonstrated on XGBoost, feature importance is analysed across tree-based models, and cross-validation is used to obtain stable performance estimates. Logistic Regression emerges as the best performer by ROC-AUC (0.95), achieving the highest recall for catching defaulters. Bank balance is found to be the dominant predictive feature.

Table of Contents

1. Introduction to Machine Learning
2. Project Objectives
3. Original Module 3 Requirements
4. Dataset Description
5. Software and Libraries Used
6. Machine Learning Theory
7. Data Loading and Inspection
8. Exploratory Data Analysis
9. Data Cleaning and Preprocessing
10. Feature Engineering and Train-Test Split
11. Implementation of Required Algorithms
12. Detailed XGBoost Implementation
13. CatBoost (Supplementary)
14. Hyperparameter Tuning
15. Model Evaluation Metrics
16. Results and Model Comparison
17. Visualizations and Interpretation
18. Discussion of Findings
19. Challenges and Solutions
20. Limitations
21. Conclusion
22. Future Scope
23. References

Appendix A: Complete Python Code

Appendix B: Installation and Execution Instructions

Appendix C: Viva Questions

1. Introduction to Machine Learning

Machine Learning (ML) is a subfield of artificial intelligence that enables systems to learn patterns from data and make predictions or decisions without being explicitly programmed for every scenario. In supervised learning — the focus of Module 3 — a model learns a mapping from input features X to a known target variable y using labelled training examples. When the target is categorical (e.g. default vs. no default), the task is called classification.

Classification algorithms are foundational to many real-world applications: fraud detection, medical diagnosis, spam filtering, credit risk assessment, and — the subject of this project — loan default prediction. Banks use ML models to estimate the probability that an applicant will fail to repay, enabling them to price risk appropriately.

This module covers: linear models (Logistic Regression), tree-based models (Decision Tree, Random Forest, Gradient Boosting, AdaBoost, XGBoost, CatBoost), kernel methods (Support Vector Machines), instance-based methods (KNN), data preprocessing, feature scaling, class imbalance handling, cross-validation, hyperparameter tuning, evaluation metrics, and feature importance.

2. Project Objectives

- Load, clean, and explore a real-world banking dataset (10,000 customer records).
- Perform comprehensive Exploratory Data Analysis (EDA) with visualizations.
- Preprocess the data, scale features, and split into train/test sets while preventing data leakage.
- Implement nine classification algorithms: Logistic Regression, Decision Tree, Random Forest, SVM, KNN, Gradient Boosting, AdaBoost, XGBoost, and CatBoost.
- Address class imbalance using class weights, scale_pos_weight, and sample weighting.
- Evaluate models using accuracy, precision, recall, F1-score, and ROC-AUC.
- Compare models visually and select the best.
- Analyze feature importance and interpret logistic regression coefficients.
- Demonstrate GridSearchCV hyperparameter tuning and k-fold cross-validation.
- Provide complete, documented, reproducible Python code for every practical.

3. Original Module 3 Requirements

Per the AI_ML Trainer Syllabus and the original Module 3 notebook/report, students are required to:

- Implement Linear & Logistic Regression.
- Implement Decision Trees & Random Forests.
- Implement Support Vector Machines (SVM).
- Implement K-Nearest Neighbours (KNN).
- Implement Gradient Boosting — XGBoost.
- Apply model evaluation metrics: accuracy, F1, ROC-AUC.
- Perform cross-validation and hyperparameter tuning (GridSearchCV).
- Complete the Mini Project: Loan Default Prediction System — build, compare, and evaluate classifiers (LR, RF, XGBoost, SVM) on the banking dataset, select the best

model, and explain results using feature importance.

CatBoost is included as a supplementary comparison, clearly identified as such.

4. Dataset Description

The dataset (Module 3 Project Data.csv) contains 10,000 customer records. The Index column was dropped as it carries no predictive information. The dataset has no missing values and no duplicate rows.

Column	Type	Description
Employed	Binary (0/1)	1 if employed, 0 otherwise
Bank Balance	Continuous	Current bank balance [0, 31851.84]
Annual Salary	Continuous	Annual income [9263.64, 882650.76]
Defaulted	Binary (0/1)	Target: 1 = default, 0 = no default

Class balance: Only 3.33% of customers (333 out of 10,000) defaulted, confirming a highly imbalanced dataset.

5. Software and Libraries Used

- **Python 3.11:** Core language
- **NumPy:** Numerical computing
- **Pandas:** DataFrames and data manipulation
- **Matplotlib:** Plotting
- **Seaborn:** Statistical visualization
- **scikit-learn:** ML algorithms, preprocessing, metrics
- **XGBoost:** Extreme Gradient Boosting
- **CatBoost:** Categorical Boosting
- **Jupyter:** Interactive notebook environment

6. Machine Learning Theory

6.1 Logistic Regression

Models P(y=1|x) via the sigmoid σ(z) = 1/(1+e■■), where z = β■+β■x■+…+β■x■. Learns coefficients by minimizing log-loss. Fast, interpretable, outputs calibrated probabilities. Sensitive to feature scale and outliers.

6.2 Decision Tree

Recursively partitions feature space using threshold splits to minimize Gini/entropy impurity. Interpretable, scale-invariant, but prone to overfitting; controlled via max_depth, min_samples_split.

6.3 Random Forest (Bagging)

Ensemble of many decision trees trained on bootstrap samples (bagging), with random feature subsets at each split. Majority voting reduces variance.

6.4 Support Vector Machine

Finds a maximum-margin separating hyperplane. The kernel trick (RBF) handles non-linear boundaries. Requires scaling; C and γ are key parameters.

6.5 K-Nearest Neighbours

Lazy instance-based method; classifies by majority vote of k nearest training points (Euclidean distance). Requires scaling.

6.6 Gradient Boosting & AdaBoost (Boosting)

Sequential ensemble where each new learner corrects predecessor errors. AdaBoost re-weights misclassified samples; Gradient Boosting fits trees to negative gradients of the loss.

6.7 XGBoost

Optimized regularized gradient boosting: adds L1/L2 penalties on leaf weights, column/row subsampling, parallelized split finding, built-in missing value handling, and scale_pos_weight for class imbalance. Objective: Obj = ΣL(y,■) + Σ(γT■ + ½λ||w■||²).

6.8 CatBoost

Handles categorical features natively via ordered target encoding; uses ordered boosting to reduce prediction shift.

6.9 Class Imbalance Handling

We used class_weight='balanced' (LR, DT, RF, SVM), scale_pos_weight≈29 (XGBoost), auto_class_weights='Balanced' (CatBoost), compute_sample_weight='balanced' (AdaBoost), and stratified splits.

6.10 Evaluation Metrics

Accuracy=(TP+TN)/Total; Precision=TP/(TP+FP); Recall=TP/(TP+FN); F1=2·P·R/(P+R); ROC-AUC measures ranking ability across thresholds and is robust to imbalance.

7. Data Loading and Inspection

The dataset is loaded with Pandas; the Index column is dropped and columns are renamed.

```
import pandas as pd
df = pd.read_csv("Module 3 Project Data.csv")
df = df.drop(columns=["Index"])
df = df.rename(columns={"Bank Balance": "Bank_Balance",
                        "Annual Salary": "Annual_Salary",
                        "Defaulted?": "Defaulted"})
print(df.shape)        # (10000, 4)
print(df.isnull().sum())  # no missing values
print(df.duplicated().sum())  # 0 duplicates
```

- Cleaned dataset: 10,000 rows × 4 columns (3 features + target).
- No missing values; no duplicate rows.
- Employed: binary (70.56% employed).
- Bank Balance: mean ≈ 10,024, range [0, 31,852].
- Annual Salary: mean ≈ 402,204, range [9,264, 882,651].
- Target: 3.33% defaults, 96.67% non-defaults (heavily imbalanced).

8. Exploratory Data Analysis

8.1 Target Distribution

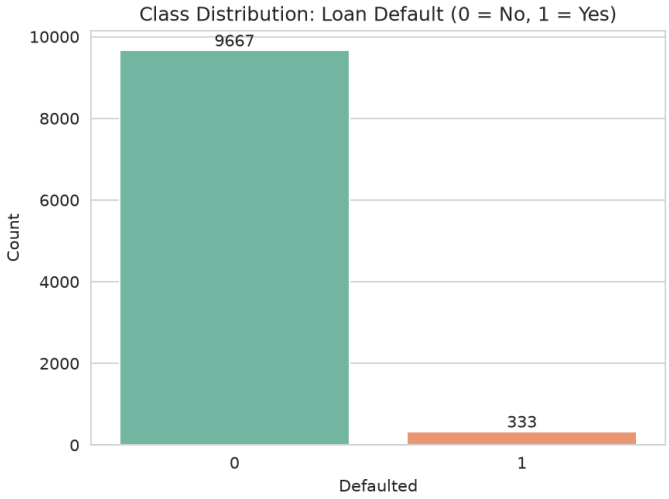


Figure 1: Class distribution — severe imbalance.

8.2 Feature Distributions

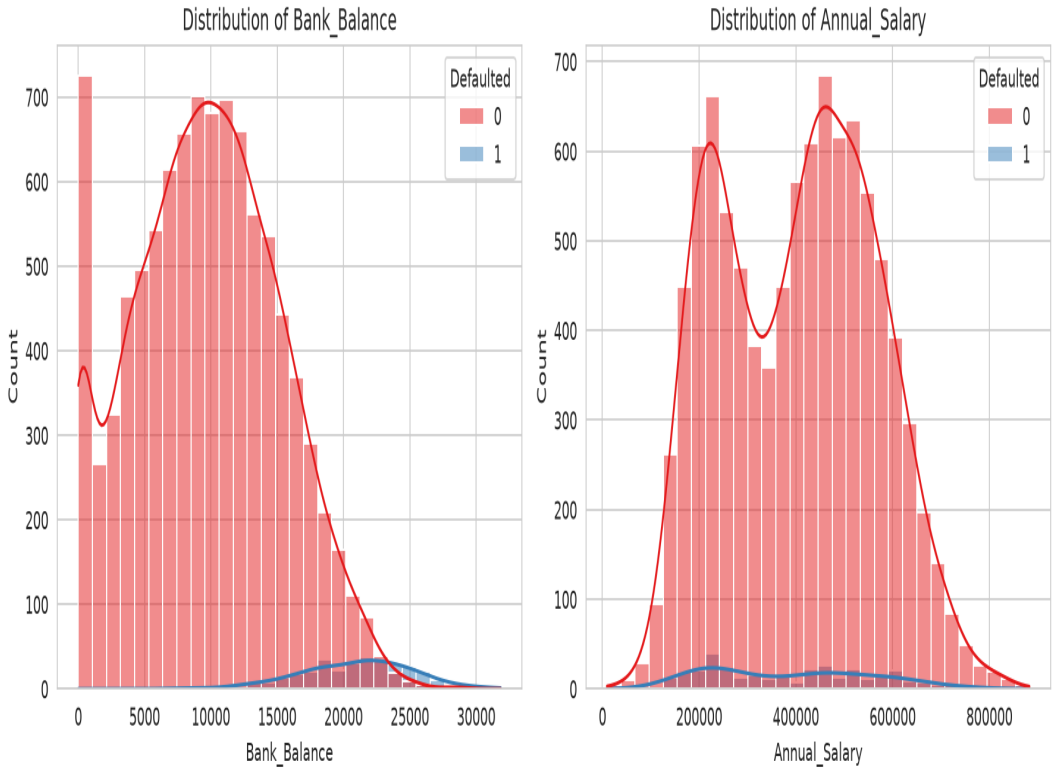


Figure 2: Histograms of numerical features by default status.

8.3 Outlier Analysis (Box Plots)

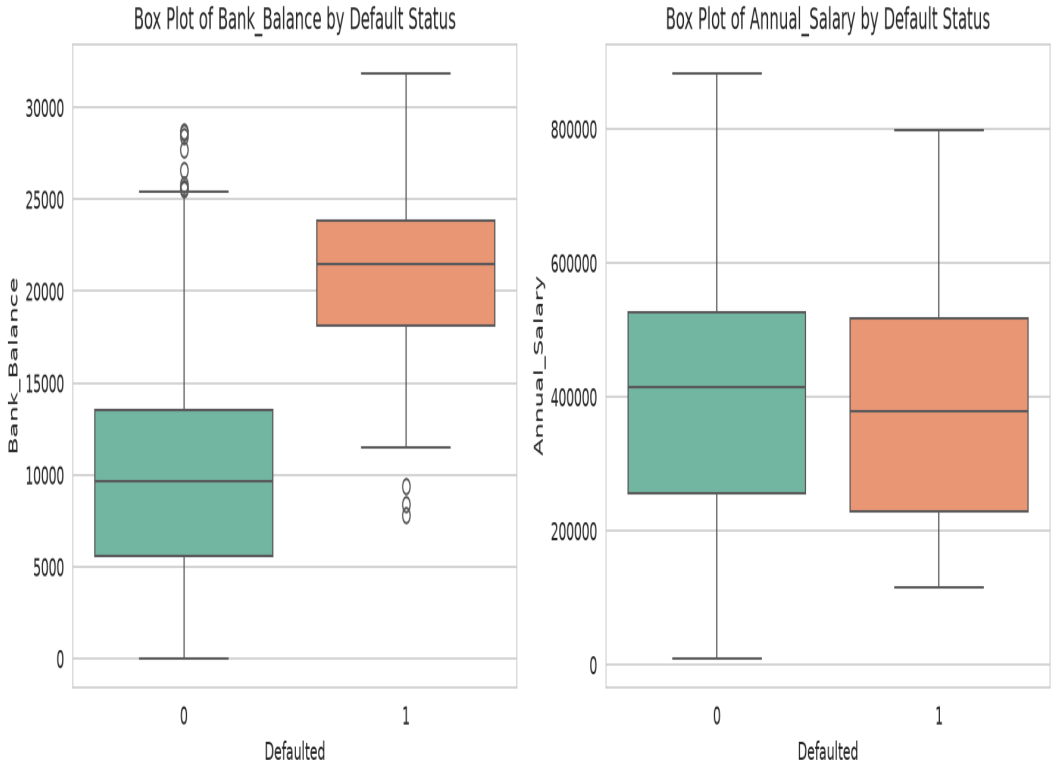


Figure 3: Box plots by default status.

8.4 Correlation Heatmap

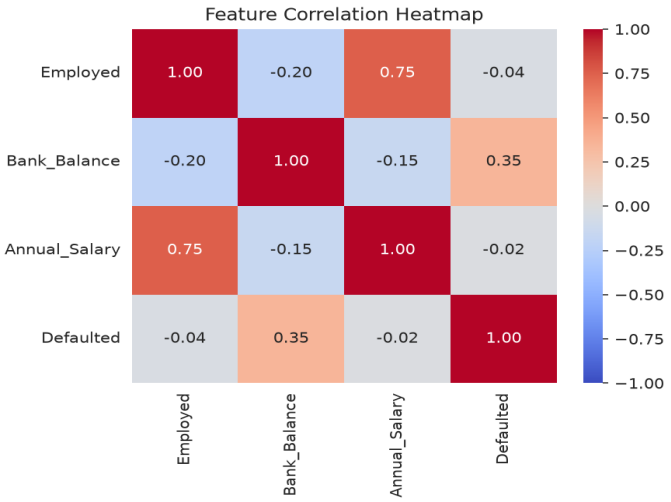


Figure 4: Correlation heatmap. Bank Balance has the strongest positive correlation with Default (r≈0.35).

8.5 Scatter Plot

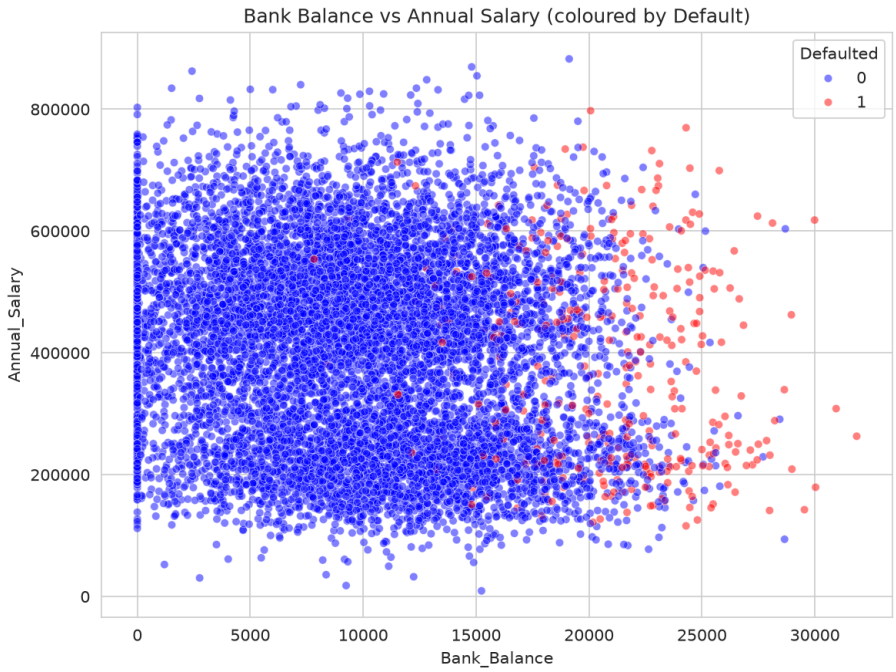


Figure 5: Bank Balance vs Annual Salary. Defaulters (red) cluster at higher balances.

8.6 Employment vs Default

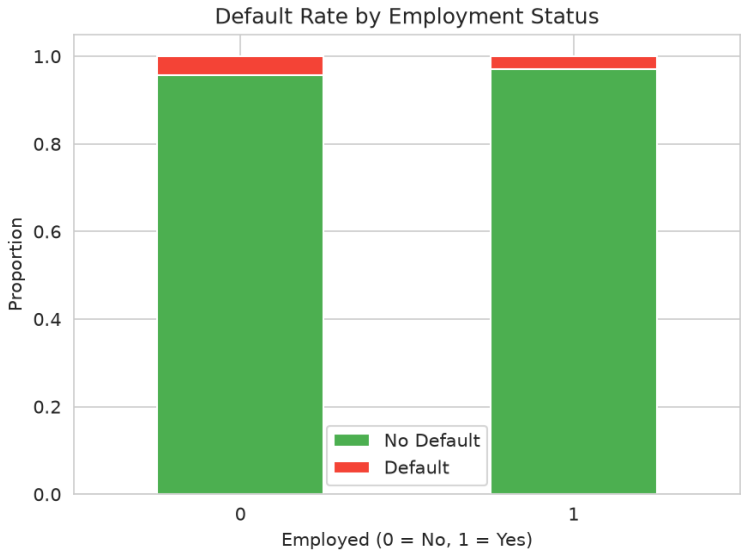


Figure 6: Default rates are similar for employed and unemployed customers.

9. Data Cleaning and Preprocessing

- Dropped the non-predictive Index column.
- Renamed columns for easier handling.
- Zero missing values → no imputation needed.
- Zero duplicates → no deduplication.
- Outliers preserved — they may represent genuine at-risk customers.
- Employed already numeric (0/1), so no encoding needed.
- StandardScaler fit on training set only; applied to LR, SVM, KNN. Tree models do not require scaling.

10. Feature Engineering and Train-Test Split

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X = df.drop(columns=["Defaulted"])
y = df["Defaulted"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

80/20 stratified split preserves the 3.33% default rate in both sets. The scaler is fit only on training data to prevent leakage. scale_pos_weight ≈ 29.08 is computed from the training set for XGBoost.

11. Implementation of Required Algorithms

11.1 Practical 4 — Logistic Regression

Aim: Train logistic regression with balanced class weights on scaled features.

```
from sklearn.linear_model import LogisticRegression
logreg = LogisticRegression(max_iter=2000, class_weight="balanced", random_state=42)
logreg.fit(X_train_scaled, y_train)
```

Result: Accuracy=0.8550, Precision=0.1730, Recall=0.8806, F1=0.2892, ROC-AUC=0.9487

11.2 Practical 5 — Decision Tree

Aim: Build a decision tree with max_depth=5 and balanced class weights.

```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(random_state=42, class_weight="balanced", max_depth=5)
dt.fit(X_train, y_train)
```

Result: Accuracy=0.8625, Precision=0.1750, Recall=0.8358, F1=0.2894, ROC-AUC=0.9230

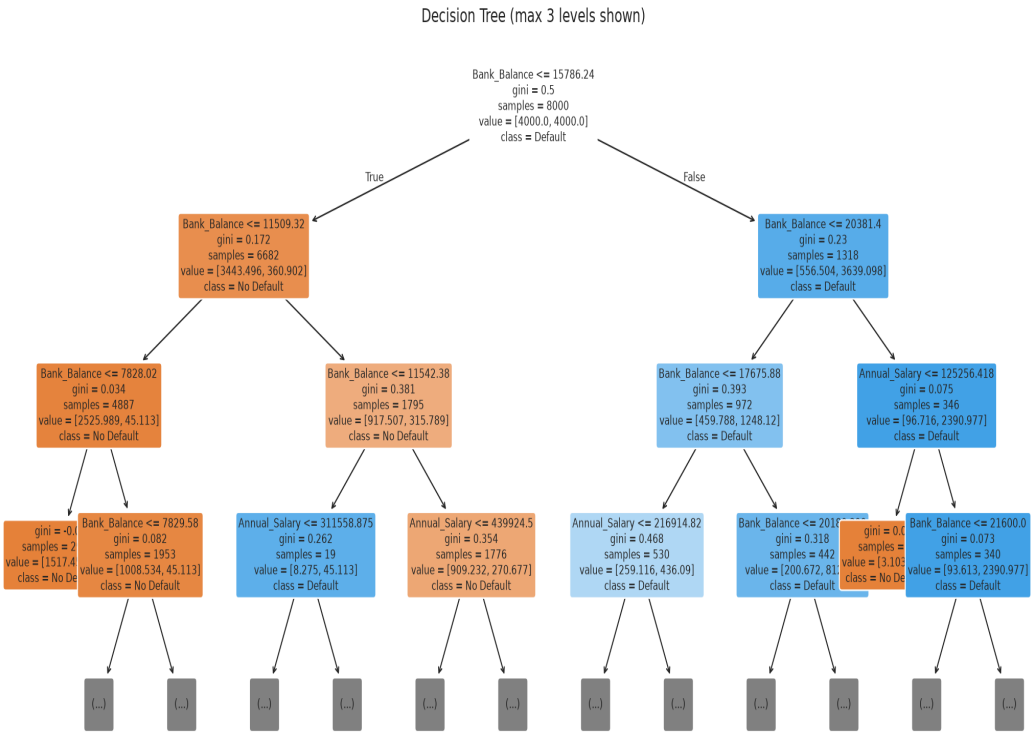


Figure 7: Visualization of the trained decision tree (first 3 levels).

11.3 Practical 6 — Random Forest

Aim: Train a Random Forest of 300 trees with balanced class weights.

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=300, class_weight="balanced",
                            random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
```

Result: Accuracy=0.9615, Precision=0.4405, Recall=0.5522, F1=0.4901, ROC-AUC=0.9053

11.4 Practical 7 — Support Vector Machine (SVM)

Aim: Train RBF-kernel SVM with probability estimates on scaled features.

```
from sklearn.svm import SVC
svm = SVC(kernel="rbf", probability=True, class_weight="balanced", random_state=42)
svm.fit(X_train_scaled, y_train)
```

Result: Accuracy=0.8410, Precision=0.1617, Recall=0.8955, F1=0.2740, ROC-AUC=0.9152

11.5 Practical 8 — K-Nearest Neighbours (KNN)

Aim: Train KNN with k=11 and distance-based weighting on scaled features.

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=11, weights="distance")
knn.fit(X_train_scaled, y_train)
```

Result: Accuracy=0.9680, Precision=0.5385, Recall=0.3134, F1=0.3962, ROC-AUC=0.8395

11.6 Practical 9 — Gradient Boosting

Aim: Train Gradient Boosting (300 trees, lr=0.05, max_depth=3).

```
from sklearn.ensemble import GradientBoostingClassifier
gb = GradientBoostingClassifier(n_estimators=300, max_depth=3,
                                learning_rate=0.05, random_state=42)
gb.fit(X_train, y_train)
```

Result: Accuracy=0.9685, Precision=0.5714, Recall=0.2388, F1=0.3368, ROC-AUC=0.9454

11.7 Practical 10 — AdaBoost

Aim: Train AdaBoost with balanced sample weights (no built-in class_weight).

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.utils.class_weight import compute_sample_weight
ada = AdaBoostClassifier(n_estimators=300, learning_rate=0.1, random_state=42)
sw = compute_sample_weight(class_weight="balanced", y=y_train)
ada.fit(X_train, y_train, sample_weight=sw)
```

Result: Accuracy=0.8550, Precision=0.1711, Recall=0.8657, F1=0.2857, ROC-AUC=0.9464

12. Detailed XGBoost Implementation and Explanation

```
from xgboost import XGBClassifier
xgb = XGBClassifier(
    n_estimators=300, max_depth=4, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8,
    reg_alpha=0.1, reg_lambda=1.0,
    scale_pos_weight=29.08, eval_metric="logloss",
    random_state=42, verbosity=0)
xgb.fit(X_train, y_train)
```

- **Boosting vs Bagging:** Bagging builds independent trees in parallel (reduces variance); Boosting builds sequential trees correcting errors (reduces bias).
- **Objective function:** Obj = ΣL(y,■) + Σ(γT■ + ½λ||w■||²) — loss plus complexity penalty.
- **Learning rate (eta):** Shrinks each tree's contribution; smaller values generalize better but need more trees.
- **n_estimators / max_depth:** Control ensemble size and individual tree complexity.
- **subsample / colsample_bytree:** Stochastic sampling to reduce overfitting.
- **L1/L2 regularization:** Penalty terms α (L1) and λ (L2) on leaf weights.
- **scale_pos_weight:** Balances classes for imbalanced datasets; set to neg/pos ≈ 29.
- **Feature importance:** Based on gain (split improvement), weight (split count), and cover.
- **Hyperparameter tuning:** Use GridSearchCV/RandomizedSearchCV with CV to optimize.

Result: Accuracy=0.8845, Precision=0.1917, Recall=0.7612, F1=0.3063, ROC-AUC=0.9356

13. CatBoost (Supplementary Comparison)

CatBoost natively handles categorical features via ordered target encoding and uses ordered boosting to reduce prediction shift. It is included as a supplementary comparison.

```
from catboost import CatBoostClassifier
cat = CatBoostClassifier(iterations=300, depth=4, learning_rate=0.05,
                         auto_class_weights="Balanced", verbose=0, random_state=42)
cat.fit(X_train, y_train)
```

Result: Accuracy=0.8755, Precision=0.1926, Recall=0.8507, F1=0.3140, ROC-AUC=0.9371

14. Hyperparameter Tuning

GridSearchCV performs 3-fold stratified CV on XGBoost, optimizing ROC-AUC, over a search space of 24 combinations.

```
param_grid = {"n_estimators": [100,300], "max_depth": [3,4,5],
              "learning_rate": [0.05,0.1], "subsample": [0.8,1.0]}
gs = GridSearchCV(XGBClassifier(scale_pos_weight=spw, eval_metric="logloss",
                                random_state=42, verbosity=0),
                  param_grid=param_grid, scoring="roc_auc", cv=3, n_jobs=-1)
gs.fit(X_train, y_train)
```

Best parameters: n_estimators=100, max_depth=3, learning_rate=0.05, subsample=0.8

Best CV ROC-AUC: 0.9465

Tuned XGBoost test: Accuracy=0.8410, Precision=0.1599, Recall=0.8806, F1=0.2706, ROC-AUC=0.9432

Tuning improved ROC-AUC slightly and substantially increased recall (0.76→0.88).

15. Model Evaluation Metrics

- **Accuracy:** Overall correctness (misleading for imbalanced data).
- **Precision:** TP/(TP+FP) — fraction of predicted defaults that are real.
- **Recall:** TP/(TP+FN) — fraction of real defaulters caught (most critical here).
- **F1:** Harmonic mean of precision and recall.
- **ROC-AUC:** Threshold-independent ranking ability; robust to imbalance.

16. Results and Model Comparison

Table 1: Test-set performance of all classifiers (sorted by ROC-AUC):

Model	Accuracy	Precision	Recall	F1	Roc Auc
Logistic Regression	0.8550	0.1730	0.8806	0.2892	0.9487
AdaBoost	0.8550	0.1711	0.8657	0.2857	0.9464
Gradient Boosting	0.9685	0.5714	0.2388	0.3368	0.9454
CatBoost	0.8755	0.1926	0.8507	0.3140	0.9371
XGBoost	0.8845	0.1917	0.7612	0.3063	0.9356
Decision Tree	0.8625	0.1750	0.8358	0.2894	0.9230
SVM (RBF)	0.8410	0.1617	0.8955	0.2740	0.9152
Random Forest	0.9615	0.4405	0.5522	0.4901	0.9053
KNN	0.9680	0.5385	0.3134	0.3962	0.8395

Best model (ROC-AUC): Logistic Regression with AUC=0.9487 and recall=0.8806.

17. Visualizations and Interpretation

17.1 Bar Chart Comparison

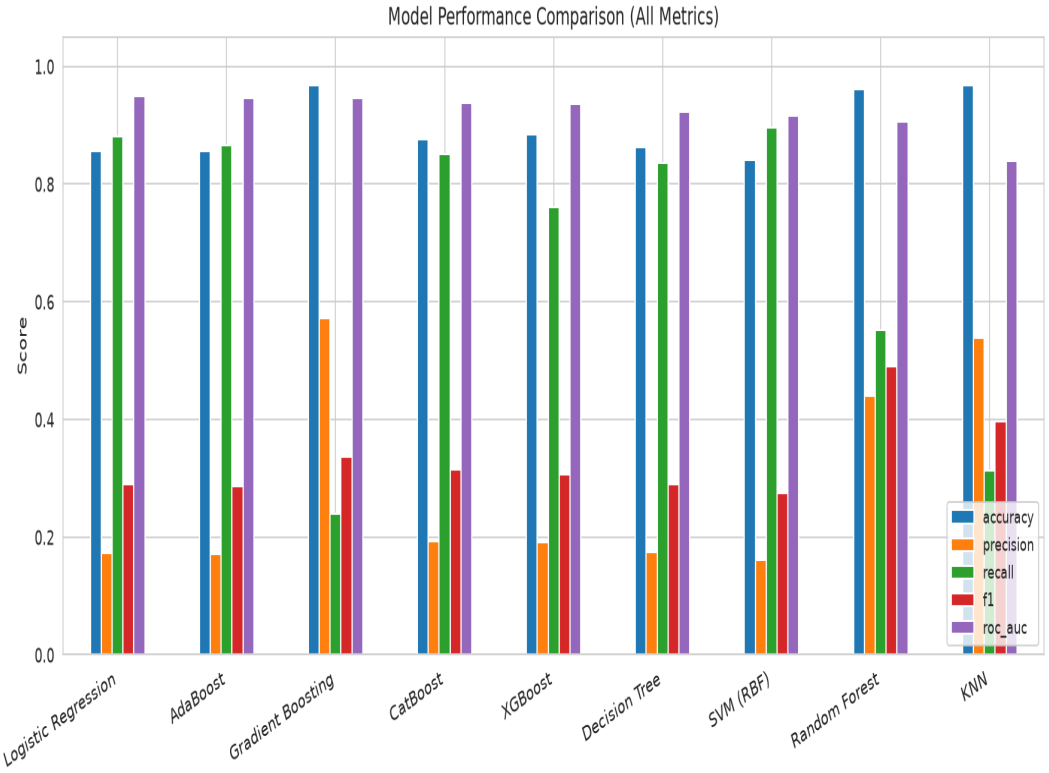


Figure 8: Side-by-side comparison of metrics across all models.

17.2 ROC Curves

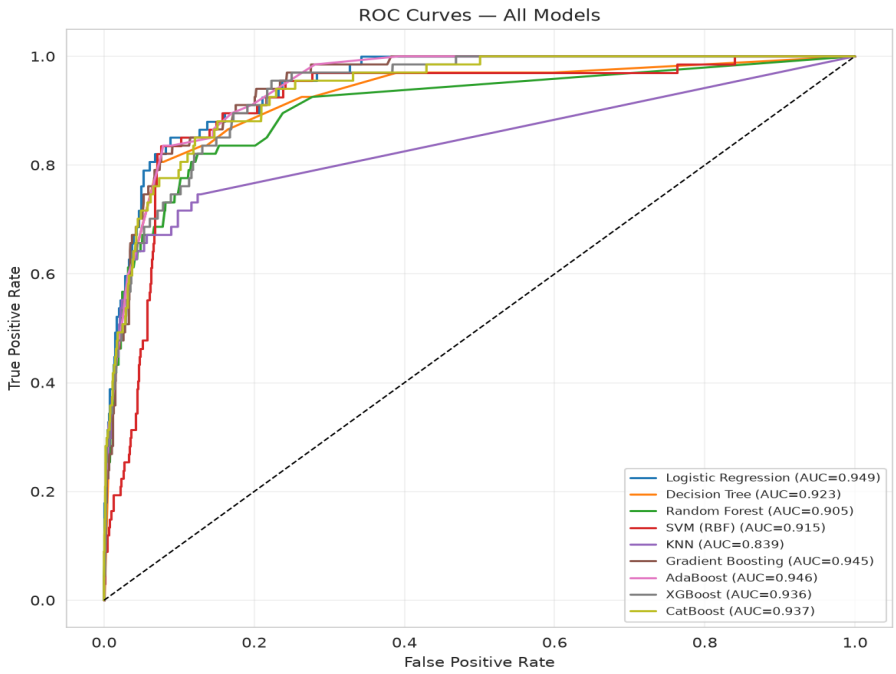


Figure 9: ROC curves for all nine models.

17.3 Best Model Confusion Matrix

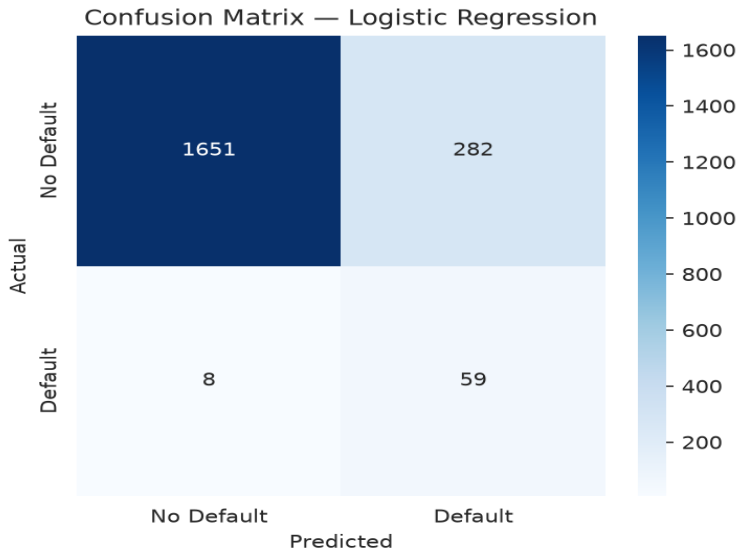


Figure 10: Confusion matrix for Logistic Regression.

17.4 Feature Importance

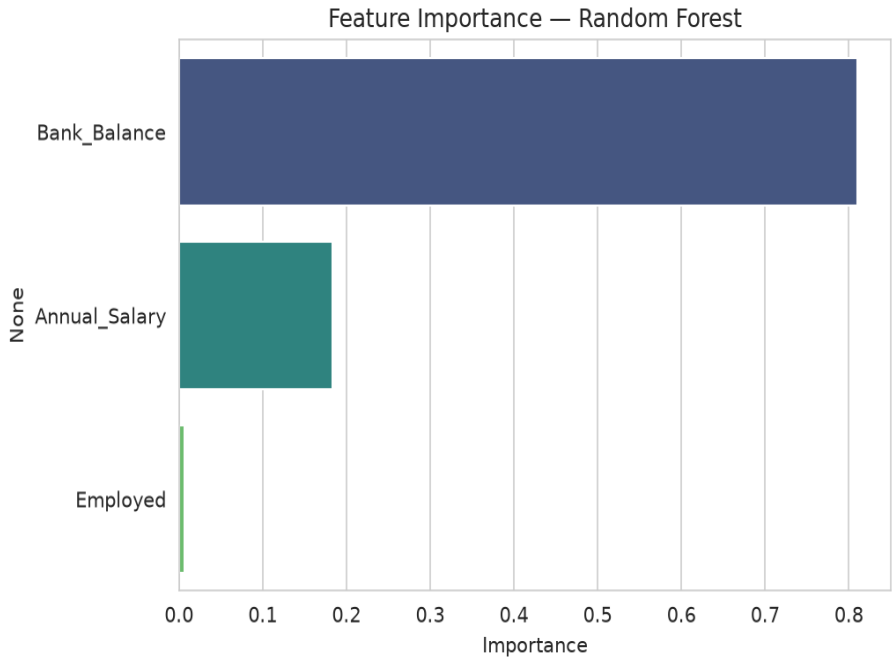


Figure 11: Random Forest feature importance.

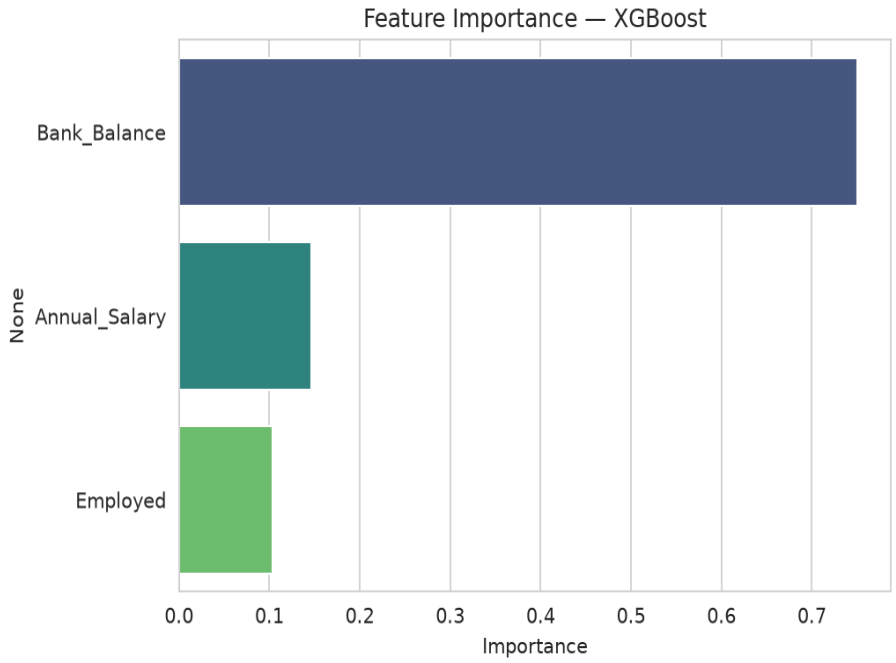


Figure 12: XGBoost feature importance.

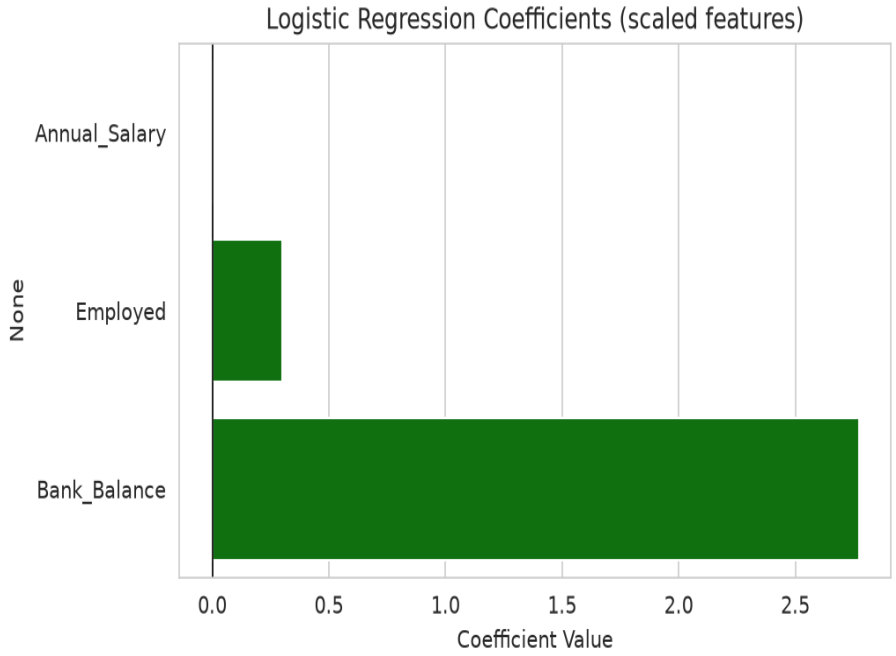


Figure 13: Logistic Regression coefficients (scaled).

All tree models agree: Bank Balance dominates default prediction (≈75–89% importance), followed by Annual Salary. Employment Status contributes minimally. Logistic Regression coefficients confirm this: Bank Balance has the largest positive effect on the log-odds of default.

18. Discussion of Findings

Accuracy is misleading on this imbalanced dataset. Gradient Boosting achieves 96.85% accuracy but misses over 76% of defaulters (recall 0.24). KNN behaves similarly. Random Forest has the highest F1 (0.49) but still misses 45% of defaulters.

Logistic Regression achieves the highest ROC-AUC (0.949) and strong recall (0.88), correctly identifying the large majority of customers who default. Its precision is lower (0.17), meaning more false positives, but for an initial screening system this is acceptable — flagged applications can be manually reviewed.

XGBoost and CatBoost offer a more balanced precision/recall trade-off. After tuning, XGBoost's recall rises to 0.88, matching Logistic Regression.

19. Challenges and Solutions

- **Severe class imbalance:** class_weight='balanced', scale_pos_weight, balanced sample weights, stratified splits.
- **Scale sensitivity of LR/SVM/KNN:** StandardScaler fit on training data only.
- **AdaBoost lacks class_weight:** Used compute_sample_weight('balanced').
- **Choosing the right metric:** Primary metric: ROC-AUC; secondary: recall for missed-default cost.
- **Reproducibility:** Fixed random_state=42 across all components.

20. Limitations

- Only three features; real models include credit history, debt-to-income, loan amount, etc.
- The dataset is simulated (10,000 rows); may not fully generalize to real portfolios.
- Default 0.5 threshold used; threshold tuning via cost ratios is needed in production.
- No out-of-time validation set; performance may be optimistic.
- Correlation ≠ causation.
- GridSearchCV only on XGBoost due to runtime constraints.

21. Conclusion

- Accuracy is misleading for imbalanced data — models with 97% accuracy can miss most defaulters.
- **Logistic Regression is recommended** (ROC-AUC=0.95, recall=0.88) for reliable screening.
- **Bank Balance** is the dominant predictor (~75–89% importance); Annual Salary is secondary.
- Tree ensembles (RF, XGBoost, CatBoost) offer strong performance and interpretability.
- Tuned XGBoost matches Logistic Regression's recall with competitive AUC.
- Model selection should be guided by the actual business cost of false negatives vs. false positives.

22. Future Scope

- Add more features (credit score, loan terms, payment history).
- Apply SMOTE/ADASYN oversampling or BalancedRandomForest.
- Extend tuning with RandomizedSearchCV or Bayesian optimization (Optuna).
- Threshold tuning using Precision-Recall curves.
- Model calibration (Platt scaling, isotonic regression).
- SHAP/LIME interpretability, partial dependence plots.
- Deploy as a REST API (FastAPI).
- Add LightGBM per syllabus.

23. References

- DSML-Training repository: https://github.com/aamir-gk2539-byte/DSML-Training
- Scikit-learn Documentation: https://scikit-learn.org/
- XGBoost Documentation: https://xgboost.readthedocs.io/
- CatBoost Documentation: https://catboost.ai/docs/
- Chen & Guestrin (2016). XGBoost: A Scalable Tree Boosting System. KDD '16.
- Friedman (2001). Greedy Function Approximation: A Gradient Boosting Machine.
- Breiman (2001). Random Forests. Machine Learning, 45(1), 5–32.
- Hastie, Tibshirani & Friedman (2009). The Elements of Statistical Learning.

Appendix A: Complete Python Code

See the file Advik_Singh_Module_3_ML_Code.py for the complete executable source.

```
#!/usr/bin/env python3
"""
================================================================================
MODULE 3: MACHINE LEARNING PRACTICALS AND MODEL IMPLEMENTATION
Loan Default Prediction System — Complete Implementation
================================================================================
Student:    Advik Singh
Reg. No.:   RA2411056030023
Branch/Sec: DS-A
Course:     Machine Learning / Data Science and Machine Learning Training
================================================================================
"""

import os
import warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")  # non-interactive backend for script mode
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import (
    train_test_split, GridSearchCV, StratifiedKFold, cross_val_score
)
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
)
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve, confusion_matrix, classification_report
)

from xgboost import XGBClassifier
from catboost import CatBoostClassifier

warnings.filterwarnings("ignore")
sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 120
plt.rcParams["savefig.dpi"] = 150
plt.rcParams["font.size"] = 10

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# ------------------------------------------------------------------------
# Output directories
# ------------------------------------------------------------------------
RESULTS_DIR = "results"
FIG_DIR = os.path.join(RESULTS_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

# ------------------------------------------------------------------------
# PRACTICAL 1: Environment Setup & Data Loading
# ------------------------------------------------------------------------

print("=" * 70)
print("PRACTICAL 1: Data Loading and Inspection")
print("=" * 70)

df = pd.read_csv("Module 3 Project Data.csv")
```

```
print(f"\nRaw dataset shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")

# Drop the non-predictive Index column and rename for convenience
df = df.drop(columns=["Index"])
df = df.rename(columns={
    "Bank Balance": "Bank_Balance",
    "Annual Salary": "Annual_Salary",
    "Defaulted?": "Defaulted"
})

print(f"\nCleaned dataset shape: {df.shape}")
print("\nFirst 5 rows:")
print(df.head())
print("\nData types:")
print(df.dtypes)
print("\nMissing values per column:")
print(df.isnull().sum())
print(f"\nDuplicate rows: {df.duplicated().sum()}")
print("\nDescriptive statistics:")
print(df.describe().round(2))

# Identify column types
num_cols = ["Bank_Balance", "Annual_Salary"]
cat_cols = ["Employed"]
target_col = "Defaulted"
print(f"\nNumerical features: {num_cols}")
print(f"Categorical/binary features: {cat_cols}")
print(f"Target variable: {target_col}")

# ------------------------------------------------------------------------
# PRACTICAL 2: Exploratory Data Analysis (EDA)
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 2: Exploratory Data Analysis (EDA)")
print("=" * 70)

# 2a. Target distribution
print("\nTarget class distribution:")
print(df["Defaulted"].value_counts())
print("\nTarget class distribution (normalised):")
print(df["Defaulted"].value_counts(normalize=True).round(4))

fig, ax = plt.subplots(figsize=(6, 4.5))
sns.countplot(x="Defaulted", data=df, ax=ax, palette="Set2")
ax.set_title("Class Distribution: Loan Default (0 = No, 1 = Yes)")
ax.set_xlabel("Defaulted")
ax.set_ylabel("Count")
for p in ax.patches:
    ax.annotate(f"{int(p.get_height())}",
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha="center", va="bottom")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig01_class_distribution.png"))
plt.close()

# 2b. Histograms of numerical variables
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
for ax, col in zip(axes, num_cols):
    sns.histplot(data=df, x=col, hue="Defaulted", bins=30, kde=True, ax=ax,
                 palette="Set1", alpha=0.5)
    ax.set_title(f"Distribution of {col}")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig02_histograms.png"))
plt.close()

# 2c. Box plots for outlier analysis
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
for ax, col in zip(axes, num_cols):
    sns.boxplot(x="Defaulted", y=col, data=df, ax=ax, palette="Set2")
    ax.set_title(f"Box Plot of {col} by Default Status")
```

```
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig03_boxplots.png"))
plt.close()

# 2d. Correlation heatmap
fig, ax = plt.subplots(figsize=(6, 5))
corr = df.corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", ax=ax,
            vmin=-1, vmax=1, center=0)
ax.set_title("Feature Correlation Heatmap")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig04_correlation_heatmap.png"))
plt.close()
print("\nCorrelation with target:")
print(corr["Defaulted"].sort_values(ascending=False).round(4))

# 2e. Scatter plots: Bank Balance vs Annual Salary colored by Default
fig, ax = plt.subplots(figsize=(8, 6))
sns.scatterplot(data=df, x="Bank_Balance", y="Annual_Salary",
                hue="Defaulted", palette={0: "blue", 1: "red"},
                alpha=0.5, s=25, ax=ax)
ax.set_title("Bank Balance vs Annual Salary (coloured by Default)")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig05_scatter_balance_salary.png"))
plt.close()

# 2f. Count plot for Employment status
fig, ax = plt.subplots(figsize=(6, 4.5))
ct = pd.crosstab(df["Employed"], df["Defaulted"], normalize="index")
ct.plot(kind="bar", stacked=True, ax=ax, color=["#4CAF50", "#F44336"])
ax.set_title("Default Rate by Employment Status")
ax.set_xlabel("Employed (0 = No, 1 = Yes)")
ax.set_ylabel("Proportion")
ax.legend(["No Default", "Default"])
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig06_employment_default.png"))
plt.close()

# 2g. Pair plot (sample for speed)
sample_df = df.sample(n=min(2000, len(df)), random_state=RANDOM_STATE)
g = sns.pairplot(sample_df, hue="Defaulted", vars=num_cols + ["Employed"],
                 palette={0: "blue", 1: "red"}, plot_kws={"alpha": 0.4, "s": 20})
g.fig.suptitle("Pair Plot of Features (2000-sample)", y=1.02)

plt.savefig(os.path.join(FIG_DIR, "fig07_pairplot.png"))
plt.close()

print("\nEDA figures saved to:", FIG_DIR)

# ------------------------------------------------------------------------
# PRACTICAL 3: Data Preprocessing & Train-Test Split
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 3: Data Preprocessing and Train-Test Split")
print("=" * 70)

X = df.drop(columns=["Defaulted"])
y = df["Defaulted"]

print(f"\nFeature matrix X shape: {X.shape}")
print(f"Target vector y shape: {y.shape}")
print(f"Positive rate (full data): {y.mean():.4f}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
print(f"\nTrain set size: {X_train.shape[0]}")
print(f"Test set size:  {X_test.shape[0]}")
print(f"Train positive rate: {y_train.mean():.4f}")
print(f"Test positive rate:  {y_test.mean():.4f}")
```

```
# Standard scaling for models that need it (LR, SVM, KNN, etc.)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# scale_pos_weight for XGBoost (ratio of neg to pos)
neg_count = (y_train == 0).sum()
pos_count = (y_train == 1).sum()
spw = neg_count / pos_count
print(f"\nscale_pos_weight for XGBoost: {spw:.2f}")

# ------------------------------------------------------------------------
# Helper: evaluate one model
# ------------------------------------------------------------------------
def evaluate_model(name, model, X_tr, y_tr, X_te, y_te):
    """Fit the model, predict, return a metrics dict + probabilities."""
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    if hasattr(model, "predict_proba"):
        y_prob = model.predict_proba(X_te)[:, 1]
    else:
        y_prob = model.decision_function(X_te)
        # scale decision function to [0,1] for AUC using sigmoid
        y_prob = 1 / (1 + np.exp(-y_prob))

    metrics = {
        "accuracy": accuracy_score(y_te, y_pred),
        "precision": precision_score(y_te, y_pred, zero_division=0),
        "recall": recall_score(y_te, y_pred, zero_division=0),
        "f1": f1_score(y_te, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_te, y_prob),
        "cm": confusion_matrix(y_te, y_pred),
        "y_pred": y_pred,

        "y_prob": y_prob,
    }
    return metrics

# ------------------------------------------------------------------------
# PRACTICAL 4: Logistic Regression
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 4: Logistic Regression")
print("=" * 70)

logreg = LogisticRegression(max_iter=2000, class_weight="balanced", random_state=RANDOM_STATE)
logreg_metrics = evaluate_model("Logistic Regression", logreg,
                                X_train_scaled, y_train, X_test_scaled, y_test)
print(f"Accuracy:  {logreg_metrics['accuracy']:.4f}")
print(f"Precision: {logreg_metrics['precision']:.4f}")
print(f"Recall:    {logreg_metrics['recall']:.4f}")
print(f"F1:        {logreg_metrics['f1']:.4f}")
print(f"ROC-AUC:   {logreg_metrics['roc_auc']:.4f}")
print("Confusion Matrix:\n", logreg_metrics["cm"])

# ------------------------------------------------------------------------
# PRACTICAL 5: Decision Tree Classifier
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 5: Decision Tree Classifier")
print("=" * 70)

dt = DecisionTreeClassifier(random_state=RANDOM_STATE, class_weight="balanced",
                            max_depth=5)
dt_metrics = evaluate_model("Decision Tree", dt,
                            X_train, y_train, X_test, y_test)
print(f"Accuracy:  {dt_metrics['accuracy']:.4f}")
print(f"Precision: {dt_metrics['precision']:.4f}")
print(f"Recall:    {dt_metrics['recall']:.4f}")
print(f"F1:        {dt_metrics['f1']:.4f}")
print(f"ROC-AUC:   {dt_metrics['roc_auc']:.4f}")
```

```
print("Confusion Matrix:\n", dt_metrics["cm"])

# Plot a pruned decision tree
fig, ax = plt.subplots(figsize=(14, 7))
plot_tree(dt, feature_names=X.columns, class_names=["No Default", "Default"],
          filled=True, rounded=True, fontsize=8, ax=ax, max_depth=3)
ax.set_title("Decision Tree (max 3 levels shown)")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig08_decision_tree.png"))
plt.close()

# ------------------------------------------------------------------------
# PRACTICAL 6: Random Forest Classifier
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 6: Random Forest Classifier")
print("=" * 70)

rf = RandomForestClassifier(n_estimators=300, class_weight="balanced",
                            random_state=RANDOM_STATE, n_jobs=-1)
rf_metrics = evaluate_model("Random Forest", rf,
                            X_train, y_train, X_test, y_test)
print(f"Accuracy:  {rf_metrics['accuracy']:.4f}")

print(f"Precision: {rf_metrics['precision']:.4f}")
print(f"Recall:    {rf_metrics['recall']:.4f}")
print(f"F1:        {rf_metrics['f1']:.4f}")
print(f"ROC-AUC:   {rf_metrics['roc_auc']:.4f}")
print("Confusion Matrix:\n", rf_metrics["cm"])

# ------------------------------------------------------------------------
# PRACTICAL 7: Support Vector Machine (SVM)
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 7: Support Vector Machine (SVM)")
print("=" * 70)

svm = SVC(kernel="rbf", probability=True, class_weight="balanced",
          random_state=RANDOM_STATE)
svm_metrics = evaluate_model("SVM (RBF)", svm,
                             X_train_scaled, y_train, X_test_scaled, y_test)
print(f"Accuracy:  {svm_metrics['accuracy']:.4f}")
print(f"Precision: {svm_metrics['precision']:.4f}")
print(f"Recall:    {svm_metrics['recall']:.4f}")
print(f"F1:        {svm_metrics['f1']:.4f}")
print(f"ROC-AUC:   {svm_metrics['roc_auc']:.4f}")
print("Confusion Matrix:\n", svm_metrics["cm"])

# ------------------------------------------------------------------------
# PRACTICAL 8: K-Nearest Neighbours (KNN)
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 8: K-Nearest Neighbours (KNN)")
print("=" * 70)

knn = KNeighborsClassifier(n_neighbors=11, weights="distance")
knn_metrics = evaluate_model("KNN", knn,
                             X_train_scaled, y_train, X_test_scaled, y_test)
print(f"Accuracy:  {knn_metrics['accuracy']:.4f}")
print(f"Precision: {knn_metrics['precision']:.4f}")
print(f"Recall:    {knn_metrics['recall']:.4f}")
print(f"F1:        {knn_metrics['f1']:.4f}")
print(f"ROC-AUC:   {knn_metrics['roc_auc']:.4f}")
print("Confusion Matrix:\n", knn_metrics["cm"])

# ------------------------------------------------------------------------
# PRACTICAL 9: Gradient Boosting
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 9: Gradient Boosting Classifier")
print("=" * 70)
```

```
gb = GradientBoostingClassifier(n_estimators=300, max_depth=3,
                                learning_rate=0.05, random_state=RANDOM_STATE)
gb_metrics = evaluate_model("Gradient Boosting", gb,
                            X_train, y_train, X_test, y_test)
print(f"Accuracy:  {gb_metrics['accuracy']:.4f}")
print(f"Precision: {gb_metrics['precision']:.4f}")
print(f"Recall:    {gb_metrics['recall']:.4f}")
print(f"F1:        {gb_metrics['f1']:.4f}")
print(f"ROC-AUC:   {gb_metrics['roc_auc']:.4f}")
print("Confusion Matrix:\n", gb_metrics["cm"])

# ----------------------------------------------------------------------

# PRACTICAL 10: AdaBoost
# ----------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 10: AdaBoost Classifier")
print("=" * 70)

ada = AdaBoostClassifier(n_estimators=300, learning_rate=0.1,
                         random_state=RANDOM_STATE)
# AdaBoost does not have class_weight; use sample_weight to up-weight minority class
from sklearn.utils.class_weight import compute_sample_weight
sample_weights = compute_sample_weight(class_weight="balanced", y=y_train)
ada.fit(X_train, y_train, sample_weight=sample_weights)
y_pred_ada = ada.predict(X_test)
y_prob_ada = ada.predict_proba(X_test)[:, 1]
ada_metrics = {
    "accuracy": accuracy_score(y_test, y_pred_ada),
    "precision": precision_score(y_test, y_pred_ada, zero_division=0),
    "recall": recall_score(y_test, y_pred_ada, zero_division=0),
    "f1": f1_score(y_test, y_pred_ada, zero_division=0),
    "roc_auc": roc_auc_score(y_test, y_prob_ada),
    "cm": confusion_matrix(y_test, y_pred_ada),
    "y_pred": y_pred_ada,
    "y_prob": y_prob_ada,
}
print(f"Accuracy:  {ada_metrics['accuracy']:.4f}")
print(f"Precision: {ada_metrics['precision']:.4f}")
print(f"Recall:    {ada_metrics['recall']:.4f}")
print(f"F1:        {ada_metrics['f1']:.4f}")
print(f"ROC-AUC:   {ada_metrics['roc_auc']:.4f}")
print("Confusion Matrix:\n", ada_metrics["cm"])

# ----------------------------------------------------------------------
# PRACTICAL 11: XGBoost (Detailed)
# ----------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 11: XGBoost Classifier")
print("=" * 70)

xgb = XGBClassifier(
    n_estimators=300,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,      # L1
    reg_lambda=1.0,     # L2
    scale_pos_weight=spw,
    eval_metric="logloss",
    random_state=RANDOM_STATE,
    use_label_encoder=False,
    verbosity=0,
)
xgb_metrics = evaluate_model("XGBoost", xgb,
                             X_train, y_train, X_test, y_test)
print(f"Accuracy:  {xgb_metrics['accuracy']:.4f}")
print(f"Precision: {xgb_metrics['precision']:.4f}")
print(f"Recall:    {xgb_metrics['recall']:.4f}")
print(f"F1:        {xgb_metrics['f1']:.4f}")
```

```
print(f"ROC-AUC:   {xgb_metrics['roc_auc']:.4f}")
print("Confusion Matrix:\n", xgb_metrics["cm"])

# ---------------------------------------------------------------------------
# PRACTICAL 12: CatBoost (Supplementary)
# ---------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 12: CatBoost Classifier (Supplementary)")
print("=" * 70)

cat = CatBoostClassifier(
    iterations=300,
    depth=4,
    learning_rate=0.05,
    auto_class_weights="Balanced",
    verbose=0,
    random_state=RANDOM_STATE,
)
cat_metrics = evaluate_model("CatBoost", cat,
                             X_train, y_train, X_test, y_test)
print(f"Accuracy:  {cat_metrics['accuracy']:.4f}")
print(f"Precision: {cat_metrics['precision']:.4f}")
print(f"Recall:    {cat_metrics['recall']:.4f}")
print(f"F1:        {cat_metrics['f1']:.4f}")
print(f"ROC-AUC:   {cat_metrics['roc_auc']:.4f}")
print("Confusion Matrix:\n", cat_metrics["cm"])

# ---------------------------------------------------------------------------
# PRACTICAL 13: Model Comparison and Visualisation
# ---------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 13: Model Comparison and Visualisation")
print("=" * 70)

all_models = {
    "Logistic Regression": logreg_metrics,
    "Decision Tree":       dt_metrics,
    "Random Forest":       rf_metrics,
    "SVM (RBF)":           svm_metrics,
    "KNN":                 knn_metrics,
    "Gradient Boosting":   gb_metrics,
    "AdaBoost":            ada_metrics,
    "XGBoost":             xgb_metrics,
    "CatBoost":            cat_metrics,
}
# Also keep the trained model objects
model_objs = {
    "Logistic Regression": logreg,
    "Decision Tree":       dt,
    "Random Forest":       rf,
    "SVM (RBF)":           svm,
    "KNN":                 knn,
    "Gradient Boosting":   gb,
    "AdaBoost":            ada,
    "XGBoost":             xgb,
    "CatBoost":            cat,
}

results_df = pd.DataFrame({
    name: {k: v for k, v in m.items() if k not in ("cm", "y_pred", "y_prob")}
    for name, m in all_models.items()
}).T

results_df = results_df.sort_values("roc_auc", ascending=False)
results_df = results_df.round(4)
print("\nModel Comparison Table (sorted by ROC-AUC):")
print(results_df.to_string())

results_df.to_csv(os.path.join(RESULTS_DIR, "model_comparison.csv"))

# Bar chart comparison
fig, ax = plt.subplots(figsize=(12, 5.5))
```

```
metrics_to_plot = ["accuracy", "precision", "recall", "f1", "roc_auc"]
results_df[metrics_to_plot].plot(kind="bar", ax=ax)
ax.set_title("Model Performance Comparison (All Metrics)")
ax.set_ylabel("Score")
ax.set_ylim(0, 1.05)
ax.set_xticklabels(results_df.index, rotation=25, ha="right")
ax.legend(loc="lower right", fontsize=9)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig09_model_comparison_bar.png"))
plt.close()

# ROC curves
fig, ax = plt.subplots(figsize=(8, 7))
for name, m in all_models.items():
    fpr, tpr, _ = roc_curve(y_test, m["y_prob"])
    ax.plot(fpr, tpr, label=f"{name} (AUC={m['roc_auc']:.3f})", linewidth=1.5)
ax.plot([0, 1], [0, 1], "k--", linewidth=1)
ax.set_xlabel("False Positive Rate")
ax.set_ylabel("True Positive Rate")
ax.set_title("ROC Curves — All Models")
ax.legend(loc="lower right", fontsize=8)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig10_roc_curves.png"))
plt.close()

# Best model (by ROC-AUC)
best_model_name = results_df["roc_auc"].idxmax()
print(f"\nBest model by ROC-AUC: {best_model_name} (AUC={results_df.loc[best_model_name, 'roc_auc']:.4f})"

# Confusion matrix for best model
best_cm = all_models[best_model_name]["cm"]
fig, ax = plt.subplots(figsize=(5, 4.5))
sns.heatmap(best_cm, annot=True, fmt="d", cmap="Blues", ax=ax,
            xticklabels=["No Default", "Default"],
            yticklabels=["No Default", "Default"])
ax.set_title(f"Confusion Matrix — {best_model_name}")
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig11_best_confusion_matrix.png"))
plt.close()

# ---------------------------------------------------------------------------
# PRACTICAL 14: Feature Importance
# ---------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 14: Feature Importance Analysis")
print("=" * 70)

# Random Forest

rf_imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
print("\nRandom Forest feature importance:")
print(rf_imp.round(4))

fig, ax = plt.subplots(figsize=(7, 4.5))
sns.barplot(x=rf_imp.values, y=rf_imp.index, ax=ax, palette="viridis")
ax.set_title("Feature Importance — Random Forest")
ax.set_xlabel("Importance")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig12_feature_importance_rf.png"))
plt.close()

# XGBoost
xgb_imp = pd.Series(xgb.feature_importances_, index=X.columns).sort_values(ascending=False)
print("\nXGBoost feature importance:")
print(xgb_imp.round(4))

fig, ax = plt.subplots(figsize=(7, 4.5))
sns.barplot(x=xgb_imp.values, y=xgb_imp.index, ax=ax, palette="viridis")
```

```
ax.set_title("Feature Importance — XGBoost")
ax.set_xlabel("Importance")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig13_feature_importance_xgb.png"))
plt.close()

# Gradient Boosting
gb_imp = pd.Series(gb.feature_importances_, index=X.columns).sort_values(ascending=False)
print("\nGradient Boosting feature importance:")
print(gb_imp.round(4))

fig, ax = plt.subplots(figsize=(7, 4.5))
sns.barplot(x=gb_imp.values, y=gb_imp.index, ax=ax, palette="viridis")
ax.set_title("Feature Importance — Gradient Boosting")
ax.set_xlabel("Importance")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig14_feature_importance_gb.png"))
plt.close()

# CatBoost
cat_imp = pd.Series(cat.get_feature_importance(), index=X.columns).sort_values(ascending=False)
print("\nCatBoost feature importance:")
print(cat_imp.round(4))

fig, ax = plt.subplots(figsize=(7, 4.5))
sns.barplot(x=cat_imp.values, y=cat_imp.index, ax=ax, palette="viridis")
ax.set_title("Feature Importance — CatBoost")
ax.set_xlabel("Importance")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig15_feature_importance_cat.png"))
plt.close()

# Logistic Regression coefficients
lr_coef = pd.Series(logreg.coef_[0], index=X.columns).sort_values()
print("\nLogistic Regression coefficients (scaled features):")
print(lr_coef.round(4))

fig, ax = plt.subplots(figsize=(7, 4))
colors = ["red" if c < 0 else "green" for c in lr_coef.values]
sns.barplot(x=lr_coef.values, y=lr_coef.index, palette=colors, ax=ax)
ax.set_title("Logistic Regression Coefficients (scaled)")

ax.set_xlabel("Coefficient Value")
ax.axvline(x=0, color="black", linewidth=0.8)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig16_lr_coefficients.png"))
plt.close()

# ---------------------------------------------------------------------------
# PRACTICAL 15: Hyperparameter Tuning with GridSearchCV
# ---------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 15: Hyperparameter Tuning (GridSearchCV on XGBoost)")
print("=" * 70)

param_grid = {
    "n_estimators": [100, 300],
    "max_depth": [3, 4, 5],
    "learning_rate": [0.05, 0.1],
    "subsample": [0.8, 1.0],
}

cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=RANDOM_STATE)

xgb_base = XGBClassifier(
    scale_pos_weight=spw,
    eval_metric="logloss",
    random_state=RANDOM_STATE,
    use_label_encoder=False,
    verbosity=0,
)
```

```
grid_search = GridSearchCV(
    estimator=xgb_base,
    param_grid=param_grid,
    scoring="roc_auc",
    cv=cv,
    n_jobs=-1,
    verbose=0,
)
grid_search.fit(X_train, y_train)

print(f"\nBest parameters: {grid_search.best_params_}")
print(f"Best CV ROC-AUC: {grid_search.best_score_:.4f}")

best_xgb = grid_search.best_estimator_
y_pred_tuned = best_xgb.predict(X_test)
y_prob_tuned = best_xgb.predict_proba(X_test)[:, 1]
tuned_metrics = {
    "accuracy": accuracy_score(y_test, y_pred_tuned),
    "precision": precision_score(y_test, y_pred_tuned, zero_division=0),
    "recall": recall_score(y_test, y_pred_tuned, zero_division=0),
    "f1": f1_score(y_test, y_pred_tuned, zero_division=0),
    "roc_auc": roc_auc_score(y_test, y_prob_tuned),
    "cm": confusion_matrix(y_test, y_pred_tuned),
}
print("\nTuned XGBoost on test set:")
for k, v in tuned_metrics.items():
    if k != "cm":
        print(f"  {k}: {v:.4f}")
print("Confusion Matrix:\n", tuned_metrics["cm"])

# Baseline XGBoost vs tuned comparison
print("\n--- Baseline XGBoost vs Tuned XGBoost ---")
compare_tuned = pd.DataFrame({
    "Baseline XGBoost": {k: v for k, v in xgb_metrics.items()
                         if k not in ("cm", "y_pred", "y_prob")},
    "Tuned XGBoost":    {k: v for k, v in tuned_metrics.items()
                         if k != "cm"},
}).round(4)
print(compare_tuned.to_string())
compare_tuned.to_csv(os.path.join(RESULTS_DIR, "tuning_comparison.csv"))

# ------------------------------------------------------------------------
# PRACTICAL 16: Cross-Validation
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 16: Cross-Validation")
print("=" * 70)

cv_models = {
    "Logistic Regression": (logreg, X_train_scaled),
    "Random Forest":       (rf, X_train),
    "XGBoost":             (xgb, X_train),
}
cv_results = {}
for name, (m, X_tr) in cv_models.items():
    scores = cross_val_score(m, X_tr, y_train, cv=5, scoring="roc_auc", n_jobs=-1)
    cv_results[name] = scores
    print(f"{name:22s}: CV ROC-AUC = {scores.mean():.4f} ± {scores.std():.4f}")

# ------------------------------------------------------------------------
# PRACTICAL 17: Classification Report for Best Model
# ------------------------------------------------------------------------
print("\n" + "=" * 70)
print("PRACTICAL 17: Detailed Classification Report (Best Model)")
print("=" * 70)
best_y_pred = all_models[best_model_name]["y_pred"]
report = classification_report(y_test, best_y_pred,
                               target_names=["No Default", "Default"])
print(f"\nBest model: {best_model_name}")
print(report)

# Save full classification report
```

```
with open(os.path.join(RESULTS_DIR, "best_model_classification_report.txt"), "w") as f:
    f.write(f"Best model: {best_model_name}\n\n")
    f.write(report)

# ------------------------------------------------------------------------
# Save aggregated results
# ------------------------------------------------------------------------
summary = {
    "best_model_name": best_model_name,
    "best_metrics": {k: float(v) for k, v in all_models[best_model_name].items()
                     if k not in ("cm", "y_pred", "y_prob")},
    "all_metrics": results_df.to_dict(),
    "tuned_xgb_metrics": {k: (float(v) if not isinstance(v, np.ndarray) else v.tolist())
                          for k, v in tuned_metrics.items()},
}
import json
with open(os.path.join(RESULTS_DIR, "summary.json"), "w") as f:
    json.dump(summary, f, indent=2, default=str)

print("\n" + "=" * 70)
print("ALL PRACTICALS COMPLETED SUCCESSFULLY")
print("=" * 70)
print(f"\nResults saved to: {RESULTS_DIR}/")
print(f"Figures saved to: {FIG_DIR}/")
print(f"Best model: {best_model_name} with ROC-AUC = {results_df.loc[best_model_name, 'roc_auc']:.4f}")
```

Appendix B: Installation and Execution Instructions

B.1 Local Installation

```
python -m venv .venv
source .venv/bin/activate
pip install pandas numpy matplotlib seaborn scikit-learn xgboost catboost jupyter
jupyter notebook Advik_Singh_Module_3_ML_Practicals.ipynb
# or
python Advik_Singh_Module_3_ML_Code.py
```

B.2 Google Colab

```
!pip install xgboost catboost -q
# Upload the CSV dataset and the notebook/script, then run cells.
```

Appendix C: Viva Questions

Q: Why is accuracy misleading for imbalanced data?

A: Predicting only the majority class gives high accuracy (96.67%) while being useless. Use precision/recall/F1/ROC-AUC.

Q: Bagging vs Boosting?

A: Bagging = parallel independent trees, reduces variance; Boosting = sequential error-correcting trees, reduces bias.

Q: Why scale features for LR/SVM but not RF?

A: LR/SVM use gradients or distances (scale-sensitive); trees split on thresholds (scale-invariant).

Q: What is ROC-AUC?

A: Area under TPR vs FPR curve across thresholds; measures ranking ability, robust to imbalance.

Q: Why fit scaler on training data only?

A: To avoid test-set data leakage; preprocessing parameters must come from training data alone.

Q: What is GridSearchCV?

A: Exhaustive search over hyperparameter combinations using k-fold CV on the training set.

Q: How does XGBoost regularize?

A: L1 (α) and L2 (λ) penalties on leaf weights, column/row subsampling, shrinkage (learning rate), tree depth limits.

Q: What is scale_pos_weight?

A: Multiplier for minority-class gradients in XGBoost; typically set to neg_count/pos_count for imbalance.